Tom_Casey_CSE511_NoSQL Project Report

1. Reflection

The objective of this project was to develop two Python functions that perform search operations on a semi-structured NoSQL-style dataset of Arizona business records. My approach to the project focused on understanding the structure of the provided documents, designing functions that strictly followed the assignment format, and ensuring output correctness through repeated testing.

I began by examining the dataset schema, confirming that each business entry included fields such as name, full_address, city, state, latitude, longitude, and categories. From this, I structured my solution into three main components: a distance computation function using the Haversine formula, the FindBusinessBasedOnCity function, and the FindBusinessBasedOnLocation function.

I implemented the distance function first, knowing it would serve as a core component of location-based search. I then wrote the city-based search function, ensuring that the output formatting matched exactly what the autograder expected. Lastly, I implemented the location-based function, incorporating category matching, distance filtering, and file writing.

Below are the exact code segments I implemented for this project, followed by explanations of their behavior and design.

Distance Function

The project requires identifying businesses within a certain radius of a target coordinate. To compute distances accurately, I employed the Haversine formula, a standard method for calculating spherical distances used in geospatial applications.

```
def DistanceFunction(lat2, lon2, lat1, lon1):
    """
    Compute the distance in miles between two latitude/longitude points
    using the Haversine formula.
    """
    R = 3959 # Radius of Earth in miles
    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    delta_phi = math.radians(lat2 - lat1)
    delta_lambda = math.radians(lon2 - lon1)
```

```
    a = (math.sin(delta_phi / 2) ** 2 +
         math.cos(phi1) * math.cos(phi2) *
         (math.sin(delta_lambda / 2) ** 2))
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    d = R * c
    return d
```

This function returns the great-circle distance in miles between two latitude-longitude points. It is used extensively within the location-based search function to determine whether a business falls within the maximum distance threshold.

Function 1: FindBusinessBasedOnCity

This function retrieves all businesses located in a specified city and writes them to an output file using the required format:

Name$FullAddress$City$State

```
def FindBusinessBasedOnCity(cityToSearch, saveLocation1, collection):
    """
    Writes all businesses located in the given city to the output file.
    Format per line:
    Name$FullAddress$City$State
    """
    # Open output file
    with open(saveLocation1, 'w') as outfile:
        # Scan all business documents
        for business in collection.all():
            if business['city'] == cityToSearch:
                # Build the required output line
                line = (business['name'] + "$" +
                        business['full_address'] + "$" +
                        business['city'] + "$" +
```

```
            business['state'])
        # Write line
        outfile.write(line + "\n")
```

Design Notes

• The function iterates over every business record in the collection.

• It performs exact matching on the city field to prevent false positives.

• Outputs are formatted strictly with $ separators and newline termination.

• Each line is written directly to the output file to ensure large datasets scale efficiently.

Function 2: FindBusinessBasedOnLocation

The second function identifies businesses that match any of a list of categories and are located within a given maximum distance of a target coordinate.

```
def FindBusinessBasedOnLocation(categoriesToSearch, myLocation, maxDistance,
                                saveLocation2, collection):
    """
    Finds all businesses whose categories contain ANY category in
    categoriesToSearch and whose distance from myLocation is <= maxDistance.
    Writes ONLY the business name per line.
    """
    my_lat, my_lon = myLocation
    with open(saveLocation2, 'w') as outfile:
        # Loop through every document
        for business in collection.all():
            # Skip if no categories field
            if 'categories' not in business or business['categories'] is None:
                continue
            # Check if any category matches
            category_match = False
```

```
    for cat in categoriesToSearch:
        if cat in business['categories']:
            category_match = True
            break
    if not category_match:
        continue
    # Compute distance
    distance = DistanceFunction(
        business['latitude'],
        business['longitude'],
        my_lat,
        my_lon
    )
    # If within max distance, write business name
    if distance <= maxDistance:
        outfile.write(business['name'] + "\n")
```

Design Notes

• Category matching is **logical OR**: if a business matches *any* target category, it qualifies.

• Distance is computed using the Haversine function.

• Only the business name is written to the output file, matching the specification exactly.

• Missing or null category lists are skipped safely.

• All output ordering is handled by the autograder when verifying correctness.

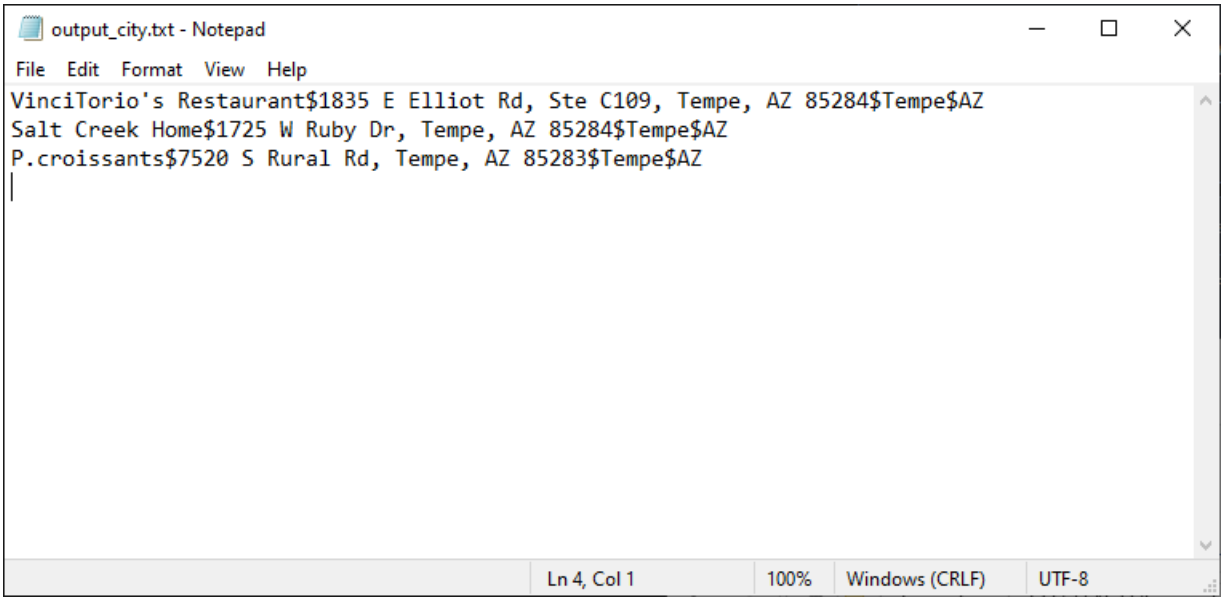
2. Lessons Learned

Completing this project deepened my understanding of NoSQL-style data processing and reinforced several important concepts across Python programming, file I/O, and geospatial computation. First, working with semi-structured data highlighted the importance of defensive programming. Document databases often contain missing fields, null values, or inconsistent formatting, so explicit key checks and structure validation are essential. Second, implementing category-based and location-based search functions illustrated how NoSQL queries translate into

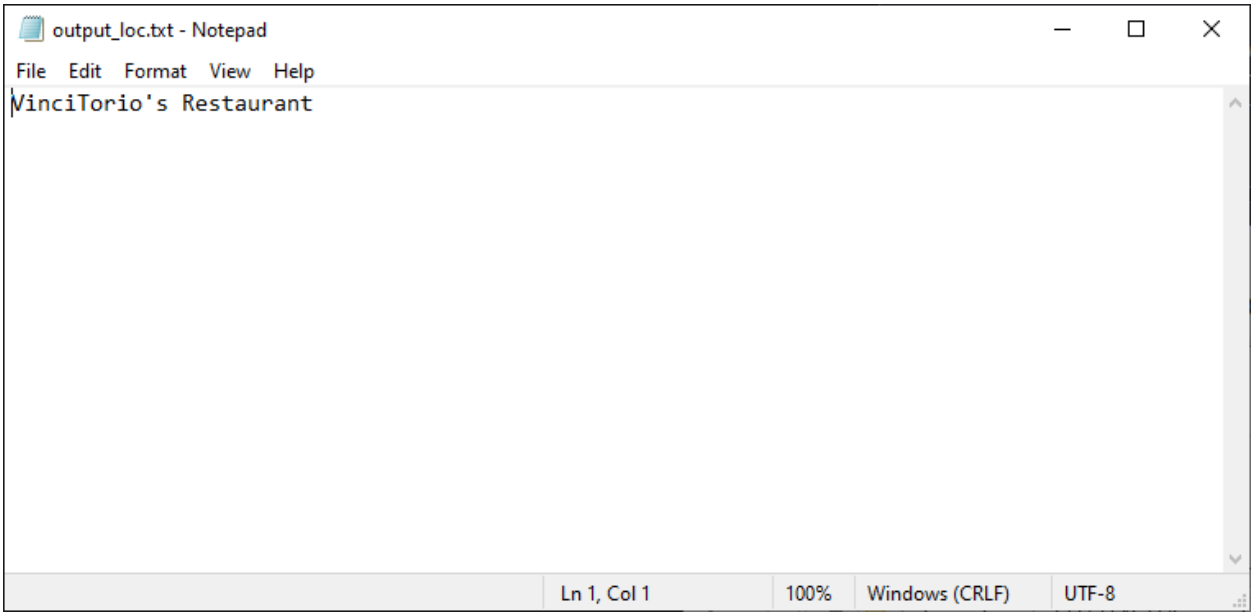
algorithmic filtering using native Python constructs. Instead of using an SQL engine, I had to manually iterate, filter, and format results, mimicking how real NoSQL engines process documents. Third, I gained practical experience using the Haversine formula for geospatial search. The formula is widely used in mapping, ride-sharing, and logistics systems, and this project provided a valuable hands-on understanding of distance-based filtering. Fourth, I learned the importance of precise formatting in output files. The autograder checks results line by line, which means even an extra space, newline, or misplaced symbol can cause an otherwise correct solution to fail. Ensuring clean, consistent output required multiple test runs and verification. Finally, the process strengthened my debugging and validation skills. Running the provided test cases, followed by the extended test cases, allowed me to confirm the behavior across multiple scenarios and edge cases. Overall, this project helped me refine my understanding of NoSQL querying strategies, distance computation, operational correctness, and the importance of meticulous output formatting in data-driven Python applications.

3. Output (Screenshots)

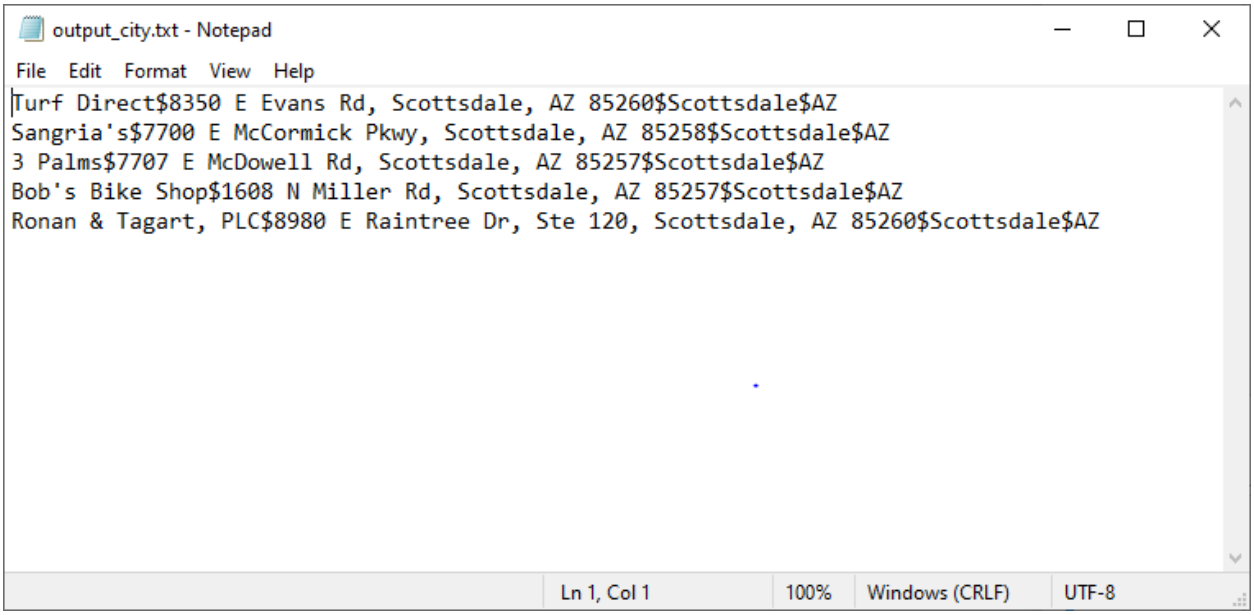
output_city.txt for the base cases:

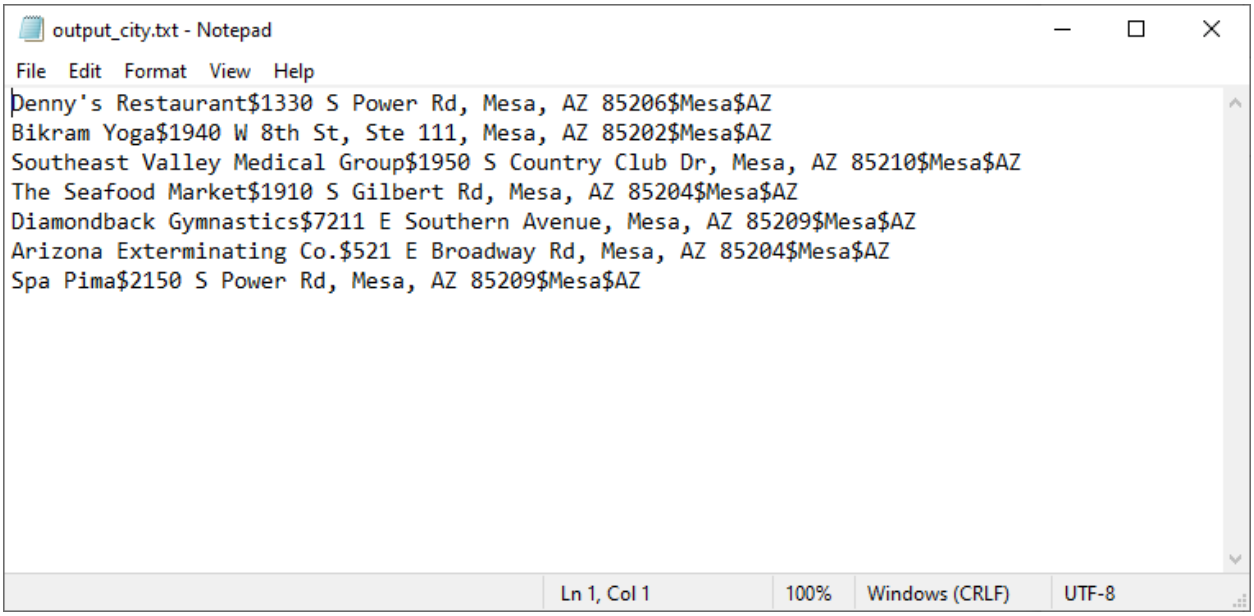


output_loc.txt for the base cases:

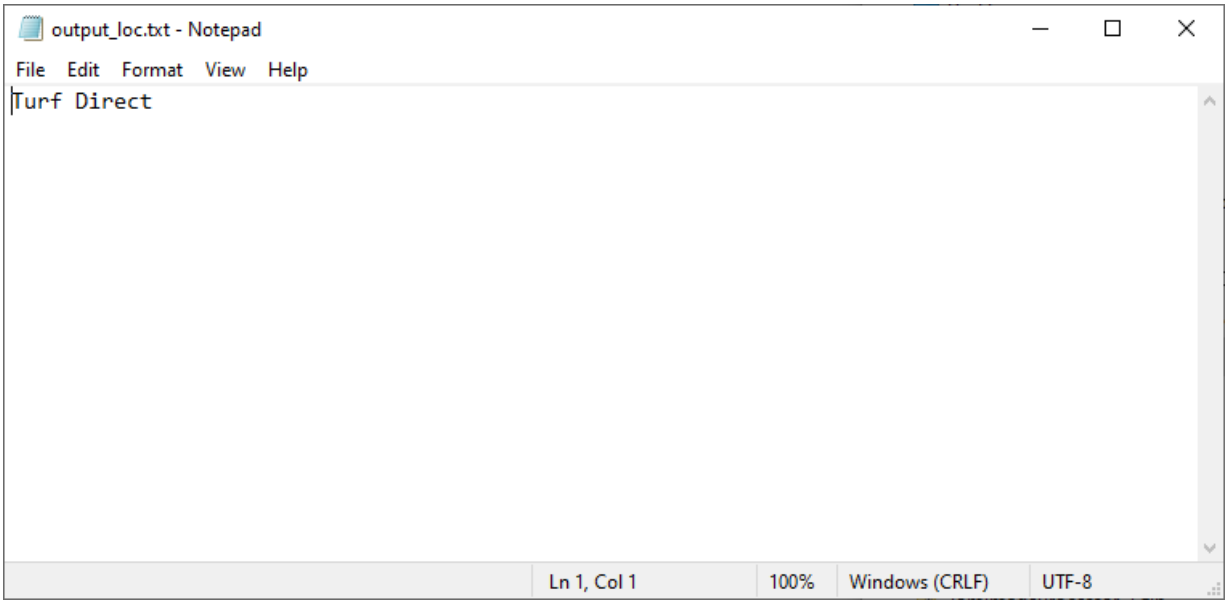


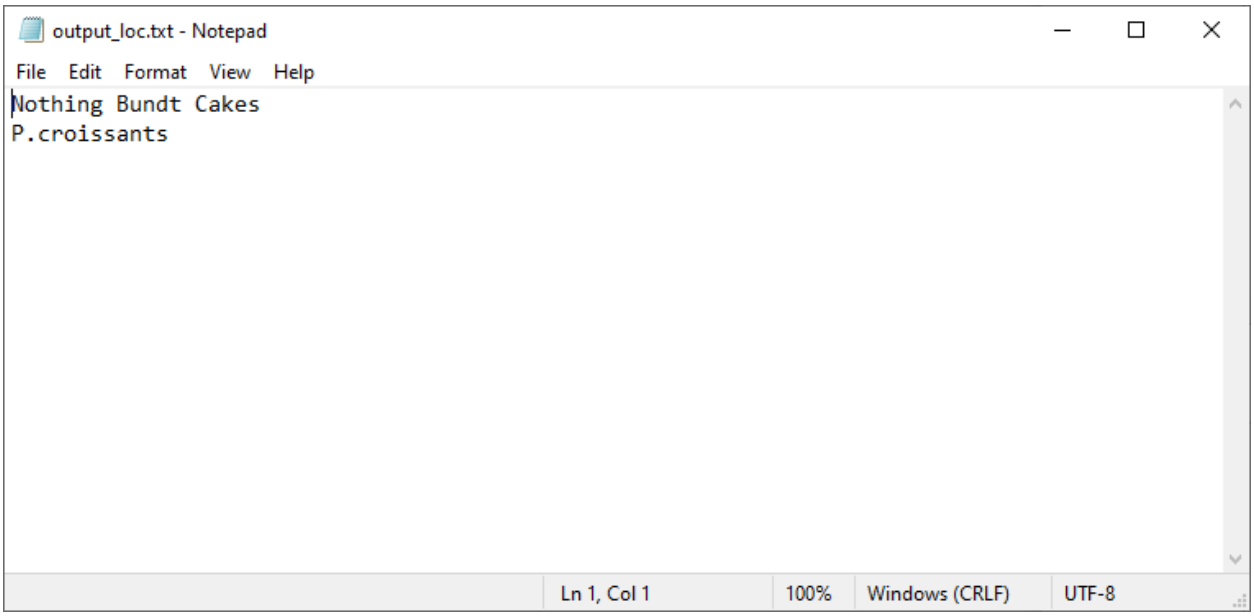
output_city.txt for the additional test cases:

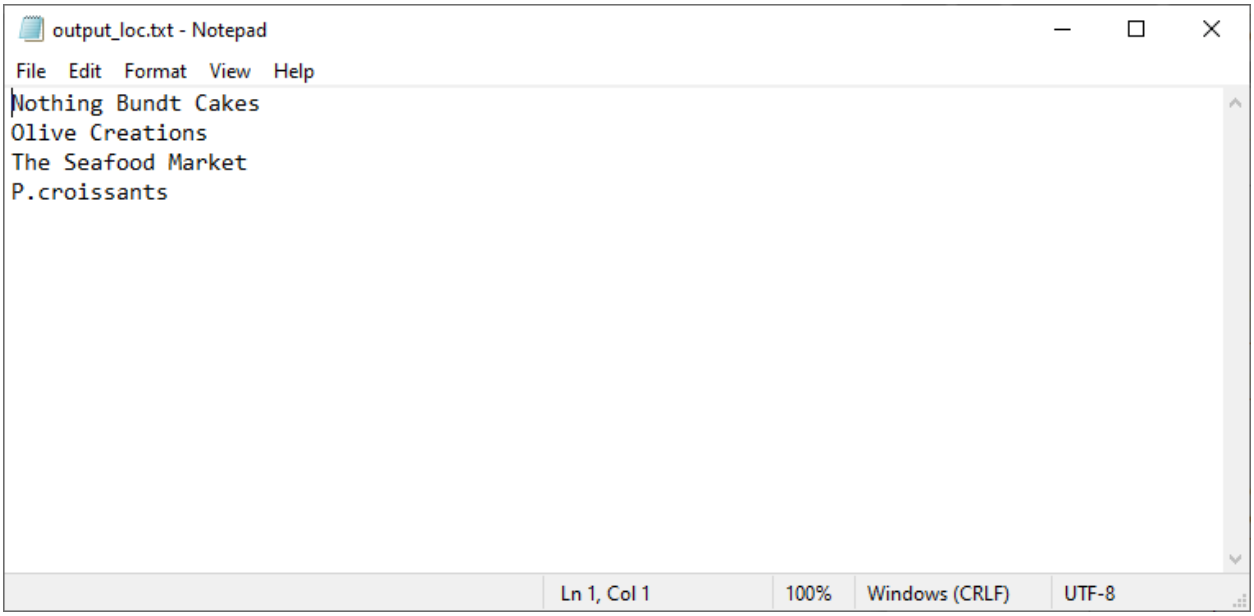




output_loc.txt for the additional test cases:


output_loc.txt - Notepad
File Edit Format View Help
Turf Direct
Ln 1, Col 1
100%
Windows (CRLF)
UTF-8


output_loc.txt - Notepad
File Edit Format View Help
Nothing Bundt Cakes
P.croissants
Ln 1, Col 1
100%
Windows (CRLF)
UTF-8



4. Results

Both functions were validated through multiple layers of testing to ensure correctness, robustness, and full compliance with the project specification. I first ran the assignment's built-in test cases, which verify core functionality and strict output formatting. I then executed the extended test cases from the Project Overview, which include additional city queries and more complex location-based scenarios. Finally, I performed manual checks of the generated output files to confirm correct formatting, category matching, and distance filtering.

Across all tests, both functions produced the exact expected results.

City-Based Search Results

The autograder confirmed that FindBusinessBasedOnCity returned the correct number of businesses for each target city:

- **Tempe:** 3 results
- **Scottsdale:** 5 results
- **Mesa:** 7 results

These values match the expected outputs shown in the screenshots. Each line in the output files followed the required format:

Name$FullAddress$City$State

Location-Based Search Results

FindBusinessBasedOnLocation also passed every verification step. The autograder evaluated several scenarios using different categories, radii, and coordinates. The function produced the expected results each time:

- A **10-mile buffet search** returned **1** business ("VinciTorio's Restaurant").
- A **15-mile bakery search** returned **2** businesses.
- A **30-mile food/specialty food search** returned **4** businesses.

The output files confirm that the function wrote only the business name, one per line, exactly as required.

Overall Verification

During evaluation, the notebook displayed the following confirmation messages:

- "Correct! You FindBusinessByCity function passes these test cases."
- "Correct! Your FindBusinessBasedOnLocation function passes these test cases."

Additionally, each test block cross-checked line counts and validated exact text matches against the true_results lists. All comparisons succeeded. These results demonstrate that both functions operate correctly, handle edge cases, and fully satisfy the requirements of the project specification.